1.3 Two approaches to scaling: Setting up the example

The rest of this chapter looks at a business chat application and the challenges when it has to scale to a large number of servers (and handle millions of simultaneous events). We'll look at the *traditional approach*, a method you're probably familiar with for building such an application (using threads and locks, RPC, and the like), and compare it to Akka's approach.

The traditional approach starts with a simple in-memory application that turns into an application that relies completely on a database for both concurrency and mutating state. Once the application needs to be more interactive, you have no choice but to poll the database. When more network services are added, the combination of working with the database and the RPC-based network increases complexity significantly. Isolating failure in this application becomes very hard as you go along. You'll probably recognize a lot of this discussion.

We'll then look at how the actor programming model simplifies the application and how Akka makes it possible to write the application once and scale it to any demand (thereby handling concurrency problems on any scale needed). Table 1.1 highlights the differences between the two approaches. As we discuss these items in the following sections, keep this overview in mind.

Table 1.1 Differences between the traditional approach and the Akka approach